Coupled Support-Size and Chart-Dimension Selection in LPS

Definitions of `auto`, `local.auto`, `full_kd`, and `sparse_kd`, with CSD5–CSD9 results

Source:
`~/current_projects/geosmooth/dev/methods/lps/status/`
`lps_coupled_support_chart_dimension_selection_report.tex`

Build 2026-07-09 09:58:23 EDT

Abstract

This report records the current meaning of the LPS support-size and chart-dimension selection policies that appear in the CSD experiments. Its main purpose is to separate two ideas that are easy to confuse. The existing `local.auto` policy estimates an anchor-specific chart-dimension field, so different anchors may use different local dimensions. The current `full_kd` and `sparse_kd` policies do not do this. They select one global pair $(k^\star, d^\star)$ for a fitted task, where k is support size and d is chart dimension. The CSD5–CSD9 experiments show that coupled global selection over (k, d) can improve over the older one-axis policies, but also that exact inner-CV minimization over the full Cartesian grid is not a solved selector. The strongest current evidence supports a prospective validation of a modest near-tie rule, not an immediate default change.

1 Purpose and executive summary

The local polynomial smoother (LPS) fits a local polynomial in a local coordinate chart around each prediction point. Two tuning choices control the geometry of each chart. The support size k controls the scale of the neighborhood, and the chart dimension d controls how many local principal directions are kept. These two quantities are statistically coupled: a larger neighborhood can support a larger local dimension, while a small neighborhood often requires a smaller dimension for numerical stability and for avoiding variance inflation.

The CSD series, short for coupled support-size and chart-dimension, was created to study this coupled selection problem. The central distinction is the following.

$$\texttt{local.auto}: \qquad d_a \text{ may vary with anchor } a,$$

whereas

$$\texttt{full\_kd} \text{ and } \texttt{sparse\_kd}: \qquad k_a = k^\star, \quad d_a = d^\star \quad \text{for all anchors } a.$$

Thus `full_kd` is not an anchor-specific automatic dimension method. It is a full Cartesian inner-CV selector over global numeric pairs (k, d).

The empirical lesson so far is mixed but useful. On the CSD6 expanded suite, `full_kd` had lower median RMSE ratio than `auto` and `local.auto` but remained meaningfully worse than the synthetic-reference candidate that minimizes known-truth outer RMSE over the full numeric (k, d) grid. This reference is defined below as the truth-facing full-grid oracle. The offline CSD9 replay suggested that a conservative one-percent near-tie rule favoring lower dimension and non-boundary candidates slightly improved over exact CV-minimum selection. The improvement was modest: the

median truth ratio moved from 1.271 to 1.252, and the count of tasks with selected truth ratio above 1.5 moved from 16 to 15. This is enough to motivate a prospective validation run, but not enough to make the rule a package default.

2 Local charts and the basic LPS object

Let $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^p$ be the observed feature cloud, and let y_i be the response associated with x_i. For an anchor a, the support neighborhood at support size k is

$$U_a(k) = \{x_a\} \cup \{\text{the } k-1 \text{ nearest neighbors of } x_a\}.$$

In the current CSD experiments, this is a nearest-neighbor support in the input geometry used by the LPS call. For a local-PCA chart, the points in $U_a(k)$ are centered at x_a and projected onto their first d principal directions. The resulting local coordinates are denoted

$$z_{a,i}^{(d)} \in \mathbb{R}^d, \qquad x_i \in U_a(k).$$

LPS then builds a local polynomial design in these coordinates. If the local polynomial degree is g, the full monomial feature count, including the intercept, is

$$q(d, g) = \binom{d+g}{g}.$$

For example, if $g = 1$, then $q(d, 1) = d + 1$. If $g = 2$, then $q(d, 2)$ includes the intercept, all linear terms, all squared terms, and all cross terms.

The local fit at anchor a is a weighted local regression problem. Its exact numerical backend can be monomial, weighted-QR, or orthogonal-polynomial stabilized, but conceptually it estimates local coefficients $\widehat{\beta}_a$ from weighted observations in $U_a(k)$ and returns the fitted value at the anchor. With centered coordinates, the anchor prediction is the fitted intercept.

3 Feasible numeric pairs

A numeric candidate (k, d) must leave enough observations to fit the local design. CSD uses the conservative prefit feasibility rule

$$k \geq q(d, g) + m,$$

where $m \geq 0$ is a design margin. In the CSD5–CSD9 experiments,

$$g = 1, \qquad m = 2, \qquad k \in \{15, \dots, 35\}, \qquad d \in \{1, \dots, 8\}.$$

Since $q(d, 1) = d + 1$, every planned pair in this grid is feasible:

$$d + 1 + 2 \leq 11 \leq 15 \qquad \text{for all } d \leq 8.$$

This rule is intentionally only a prefit screen. A candidate can still fail inside the numerical fit if the realized weighted design is rank-deficient or ill-conditioned.

4 Candidate selection and synthetic evaluation

This section defines the selection objects used throughout the report. It is included because the CSD labels are otherwise too easy to read as implementation shorthand.

An *outer task* is one fitted comparison unit. In CSD5–CSD9, an outer task is determined by a synthetic dataset, a random repetition, and an outer cross-validation fold. Let T_s denote the outer-training indices for task s, and let H_s denote the outer-heldout indices. Candidate selection is performed using only the observed responses on T_s. The known synthetic truth f_i is used only later, for auditing.

Inside T_s, the LPS fitting code creates inner validation folds. For an inner fold ℓ, let $V_{s\ell} \subset T_s$ be the validation indices and $T_{s\ell} = T_s \setminus V_{s\ell}$ be the corresponding inner-training indices. For a candidate θ, LPS is fit on $T_{s\ell}$ and predicts the held-out responses at $V_{s\ell}$. The inner-CV score is

$$C_s(\theta) = \left\{ \frac{\sum_\ell \sum_{i \in V_{s\ell}} \left(y_i - \widehat{f}^{(-\ell)}_{s,\theta}(x_i) \right)^2}{\sum_\ell |V_{s\ell}|} \right\}^{1/2}.$$

Here $\widehat{f}^{(-\ell)}_{s,\theta}$ is the LPS fit trained without the responses in $V_{s\ell}$. The selected candidate is

$$\widehat{\theta}_s = \arg\min_{\theta \in \mathcal{A}_s} C_s(\theta),$$

where $\mathcal{A}_s$ is the candidate set for the selector being used. If two candidates are exactly tied, the implementation uses deterministic candidate ordering. In practice exact ties are uncommon except in deliberately simplified smoke cases.

The meaning of θ depends on the selector. For `auto` and `local.auto`, the candidate is the support size k, because chart dimensions are resolved by the automatic dimension rule. Thus

$$\mathcal{A}_s^{\text{auto}} = \{15, 16, \ldots, 35\}, \qquad \theta = k.$$

For `full_kd` and `sparse_kd`, the candidate is a numeric pair

$$\theta = (k, d),$$

where k is support size and d is a scalar chart dimension used by all anchors in that fit.

After $\widehat{\theta}_s$ is selected, LPS is refit on all of T_s using that candidate. The fitted model is then evaluated on H_s. In real applications, one would report an outer observed prediction score against y_i. In the CSD synthetic experiments, the main audit score is truth RMSE against the known synthetic function:

$$R_{s,m} = \left\{ \frac{1}{|H_s|} \sum_{i \in H_s} \left(\widehat{f}_{s,m}(x_i) - f_i \right)^2 \right\}^{1/2},$$

where m is the selector label and $\widehat{f}_{s,m}$ is the final fit selected by that method.

The *truth-facing full-grid oracle* is a synthetic-only reference. For each outer task s, we imagine fitting every feasible numeric pair $(k, d) \in \mathcal{G}_{\text{full}}$, evaluating each one against the known truth on H_s, and keeping the best truth RMSE:

$$R_s^\star = \min_{(k,d) \in \mathcal{G}_{\text{full}}} \left\{ \frac{1}{|H_s|} \sum_{i \in H_s} \left(\widehat{f}_{s,k,d}(x_i) - f_i \right)^2 \right\}^{1/2}.$$

It is an *oracle* because it uses f_i, which is unavailable in real data. It is *truth-facing* because it minimizes truth error rather than inner CV error. It is *full-grid* because the minimization is over the complete numeric grid $\mathcal{G}_{\text{full}}$, not over a sparse candidate subset. A deployable selector such as `full_kd` may search the same grid, but it must select by $C_s(k,d)$ rather than by $R_s(k,d)$.

The main relative accuracy diagnostic is

$$\kappa_{s,m} = \frac{R_{s,m}}{R_s^\star + \epsilon}, \qquad \epsilon = 10^{-12}.$$

A value $\kappa_{s,m} = 1$ means that method m matched the truth-facing full-grid oracle on that outer task. A value 1.25 means the selected fit has 25 percent larger truth RMSE than the oracle. A value 2 means twice the oracle truth RMSE.

5 Automatic chart-dimension rules

The automatic dimension policies are based only on the observed feature cloud X, the support size k, and the local polynomial degree g. They do not use the response y and do not use the synthetic truth f.

At an anchor a, form the local-PCA singular values of the centered support $U_a(k)$:

$$\sigma_{a1} \geq \sigma_{a2} \geq \cdots \geq 0.$$

Let $e_{ar} = \sigma_{ar}^2$ be the local spectral energy. The implementation computes two candidate dimensions. The cumulative-energy dimension is the first r satisfying

$$\frac{\sum_{j=1}^{r} e_{aj}}{\sum_{j\geq 1} e_{aj}} \geq \tau_{\text{var}},$$

with default variance threshold $\tau_{\text{var}} = 0.95$. The eigengap dimension is the index r at which the ratio

$$\frac{\sigma_{ar}}{\max(\sigma_{a,r+1}, \epsilon_{\text{sv}})}$$

is largest, provided that the largest ratio is at least the default eigengap threshold $\tau_{\text{gap}} = 4$. If the eigengap rule does not find a sufficiently large gap, it is ignored. The anchor-level automatic dimension is then

$$\widehat{d}_a(k) = \begin{cases} \min\{d_a^{\text{var}}(k), d_a^{\text{gap}}(k)\}, & \text{if the eigengap dimension is available,} \\ d_a^{\text{var}}(k), & \text{otherwise.} \end{cases}$$

The result is clipped by the local design feasibility cap. For a support with k points, the cap is the largest integer r such that

$$q(r,g) = \binom{r+g}{g} \leq k - 1.$$

The subtraction by one is conservative: it leaves one degree of freedom beyond the raw number of local polynomial columns. This cap prevents the automatic dimension rule from requesting more local polynomial columns than the support can stably fit.

The `local.auto` policy uses this anchor-level value directly:

$$d_a = \widehat{d}_a(k).$$

The `auto` policy pools the same anchor-level diagnostics into one global dimension. In the current implementation, the global automatic dimension is the median of the successful anchor-level values, again clipped to the feasible range. For speed, the global `auto` helper uses at most 60 representative anchors when n is larger than 60, while `local.auto` evaluates all anchors because it needs a dimension value for each one. The pooled global dimension is

$$\widehat{d}_{\mathrm{auto}}(k) = \mathrm{median}_{a:\widehat{d}_a(k)\ \mathrm{available}}\, \widehat{d}_a(k),$$

after integer conversion and feasibility clipping. If no anchor-level estimate is available, the code uses a conservative fallback dimension allowed by the support-size and degree constraints.

6 The four selector families

This section defines the four policy labels used in the CSD reports. The most important axis is whether the policy chooses a global dimension, an anchor-specific dimension field, or a global numeric pair.

6.1 `auto`: one global automatic chart dimension

The `auto` policy asks the package to estimate a single chart dimension from the observed input geometry. Because the automatic dimension depends on the support size, write this value as

$$\widehat{d}_{\mathrm{auto}}(k).$$

Once it is resolved, the same chart dimension is used at every anchor:

$$d_a = \widehat{d}_{\mathrm{auto}}(k) \qquad \text{for all anchors } a.$$

The support-size candidate set in CSD5–CSD9 is

$$\mathcal{K} = \{15, 16, \ldots, 35\}.$$

For each $k \in \mathcal{K}$, the code resolves $\widehat{d}_{\mathrm{auto}}(k)$ from the local-PCA spectra of X, fits LPS in inner cross-validation using the global dimension $d_a = \widehat{d}_{\mathrm{auto}}(k)$, and computes the inner-CV score

$$C_s^{\mathrm{auto}}(k) = C_s\Big(k, \widehat{d}_{\mathrm{auto}}(k)\Big).$$

The selected support size is

$$\widehat{k}_s^{\mathrm{auto}} = \arg\min_{k\in\mathcal{K}} C_s^{\mathrm{auto}}(k).$$

The final outer-training fit then uses

$$k_a = \widehat{k}_s^{\mathrm{auto}}, \qquad d_a = \widehat{d}_{\mathrm{auto}}\Big(\widehat{k}_s^{\mathrm{auto}}\Big) \qquad \text{for all anchors } a.$$

Thus, in the CSD comparison, `auto` is a one-dimensional CV search over support size, with a geometry-derived global chart dimension attached to each candidate support size.

This policy is deployable and simple, but it assumes that a single working local dimension is adequate for the whole dataset. That is plausible for homogeneous manifold-like data and less plausible for stratified or non-manifold data, such as simplex-boundary geometries or unions of pieces with different intrinsic dimensions.

6.2 `local.auto`: anchor-specific automatic chart dimensions

The `local.auto` policy estimates chart dimension separately by anchor:

$$d_a = \widehat{d}_a(k).$$

The same local spectral rule from the previous section is applied at each anchor and at each candidate support size. For a fixed k, the local polynomial model at anchor a is fit in $\mathbb{R}^{\widehat{d}_a(k)}$.

The support size is not anchor-specific in the current CSD setup. The candidate set is again $\mathcal{K} = \{15, \ldots, 35\}$. For each $k \in \mathcal{K}$, the inner-CV fit uses

$$k_a = k, \qquad d_a = \widehat{d}_a(k),$$

and computes a score denoted $C_s^{\text{local.auto}}(k)$. The selected support size is

$$\widehat{k}_s^{\text{local.auto}} = \arg\min_{k \in \mathcal{K}} C_s^{\text{local.auto}}(k).$$

The final fitted model uses one selected support size for all anchors but keeps anchor-specific chart dimensions:

$$k_a = \widehat{k}_s^{\text{local.auto}}, \qquad d_a = \widehat{d}_a\left(\widehat{k}_s^{\text{local.auto}}\right).$$

This policy is the one that should be described as anchor-specific. It can adapt to geometries where some regions look one-dimensional and other regions look two- or three-dimensional. Its cost is that the local designs have varying numbers of columns, and its statistical risk is that noisy local dimension estimates can create unstable chart fields.

6.3 `full_kd`: full global Cartesian selector over numeric pairs

The `full_kd` policy defines a full Cartesian candidate set

$$\mathcal{G}_{\text{full}} = \{15, 16, \ldots, 35\} \times \{1, 2, \ldots, 8\},$$

after feasibility filtering. For each candidate $(k, d) \in \mathcal{G}_{\text{full}}$, LPS is fit with

$$k_a = k, \qquad d_a = d \qquad \text{for all anchors } a.$$

The inner-CV score for a candidate is denoted

$$C_s(k, d),$$

where s indexes the outer task, meaning a particular dataset, repetition, and outer fold. To compute this number, the code repeats the same fold procedure described above: fit on each inner-training subset, predict the corresponding inner-validation subset, square the validation residuals, and average them over all inner validation points. The current exact CV-minimum version of `full_kd` selects

$$(\widehat{k}_s, \widehat{d}_s) = \arg\min_{(k,d) \in \mathcal{G}_{\text{full}}} C_s(k, d).$$

The selected pair is global for task s. It is not a vector $\{(k_a, d_a) : a = 1, \ldots, n\}$. Once selected, all anchors in that fitted model use the same support size and the same numeric chart dimension.

In the CSD experiments, `full_kd` has two roles. First, it is a computationally expensive but auditable candidate selector using inner CV. Second, when scored against known synthetic truth across all pairs, it provides a truth-facing reference grid. These roles must not be confused. The deployable selector can only use $C_s(k, d)$; the truth-facing oracle uses $R_s(k, d)$ and is available only in synthetic experiments.

6.4 `sparse_kd`: sparse global numeric selector

The `sparse_kd` policy is a cheaper approximation to `full_kd`. It uses a smaller candidate set

$$\mathcal{G}_{\text{sparse}} \subset \mathcal{G}_{\text{full}}.$$

In the CSD5 and CSD6 runs, the sparse grid was constructed mechanically from the same planned grids. The support skeleton retained the minimum, middle, and maximum planned support sizes:

$$\mathcal{K}_{\text{sparse}} = \{15, 25, 35\}.$$

The dimension skeleton retained the low dimensions 1 and 2 and the largest planned dimension available after feasibility clipping:

$$\mathcal{D}_{\text{sparse}} = \{1, 2, 8\}.$$

Their Cartesian product produced

$$\mathcal{G}_{\text{sparse}} = \mathcal{K}_{\text{sparse}} \times \mathcal{D}_{\text{sparse}},$$

so 9 candidates were evaluated rather than the 168 candidates in $\mathcal{G}_{\text{full}} = \{15, \ldots, 35\} \times \{1, \ldots, 8\}$. As with `full_kd`, a selected sparse candidate is a single global pair:

$$(\widehat{k}_s, \widehat{d}_s) \in \mathcal{G}_{\text{sparse}}.$$

Every anchor in the fitted model then uses this same selected pair.

The engineering motivation for `sparse_kd` is PCA-coordinate reuse. For a fixed support size and kernel, the package can build local PCA coordinates once at the largest requested numeric dimension and reuse leading columns for smaller numeric dimensions. This makes numeric chart-dimension grids far cheaper than rebuilding every chart from scratch. The statistical risk is that a sparse grid can miss the good region of the full grid, so it must be benchmarked against `full_kd` before being treated as a routine selector.

7 How to read the CSD tables

The CSD tables below use the notation just defined. The column called RMSE ratio is the median of $\kappa_{s,m}$ over matched outer tasks. The column called relative regret is

$$100\,(\kappa_{s,m} - 1).$$

Thus, relative regret is measured in percent above the truth-facing full-grid oracle. A relative regret of 0 percent means the selected method matched the oracle on that task; 25 percent means the selected truth RMSE was 1.25 times the oracle truth RMSE.

8 What the CSD5–CSD9 experiments show

The CSD experiments are not one experiment repeated under different names. They answer a sequence of increasingly specific questions.

Table 1: CSD5 smoke summary. The RMSE ratio is the median selected truth RMSE divided by the truth-facing full-grid oracle on matched outer tasks. Candidate and PCA counts are medians per outer fit.

Strategy	RMSE ratio	Relative regret (%)	Candidates	PCA builds
`auto`	2.022	102.2	21	21
`local.auto`	2.114	111.4	21	21
`sparse_kd`	1.242	24.2	9	3
`full_kd`	1.334	33.4	168	21

8.1 CSD5: first coupled-grid smoke

CSD5 compared `auto`, `local.auto`, `sparse_kd`, and `full_kd` on a compact smoke suite. Table 1 gives the median strategy summaries. The main signal was that both numeric coupled selectors did better than the one-axis automatic policies in this small run, and `sparse_kd` was surprisingly competitive while using far fewer candidates.

Because CSD5 was small, its main value was not the precise ranking. Its value was showing that coupled (k, d) search was worth testing more seriously and that PCA reuse made sparse numeric grids cheap enough to consider.

8.2 CSD6: expanded relative-regret evaluation

CSD6 expanded the geometry suite to 8 datasets, 2 repetitions, and 3 outer folds, for 48 matched outer tasks per strategy. Table 2 summarizes the median results. In this broader run, `full_kd` had the lowest median RMSE ratio, while `sparse_kd` remained close and much cheaper.

Table 2: CSD6 expanded summary. The full Cartesian grid evaluated 168 numeric (k, d) candidates per task, while the sparse grid evaluated 9. The lower RMSE ratio for `full_kd` indicates better median accuracy, but the ratio remains well above 1, so exact CV minimization over the full grid is not oracle-like.

Strategy	RMSE ratio	Relative regret (%)	Candidates	PCA builds
`auto`	1.374	37.4	21	21
`local.auto`	1.410	41.0	21	21
`sparse_kd`	1.298	29.8	9	3
`full_kd`	1.271	27.1	168	21

The CSD6 conclusion is therefore not that `full_kd` solves selection. It is that global coupled numeric selection improves the median picture relative to the older automatic one-axis policies, but still leaves substantial selection regret.

8.3 CSD7: diagnosing failures

CSD7 asked why the CSD6 selectors still missed. The failure diagnostics showed that poor outcomes were not only sparse-grid misses. Some failures occurred even when `full_kd` was available. In particular, exact CV-minimum selection sometimes chose a candidate far from the truth-facing best candidate even though the better candidate was present in the full grid.

This changed the next question. Before CSD7, it was tempting to ask whether the sparse grid was too sparse. After CSD7, the more important question became whether the inner-CV surface itself was selecting too aggressively among near-tied candidates or boundary candidates.

8.4 CSD8: candidate-level CV surface audit

CSD8 persisted the full candidate-level inner-CV surface and joined it to the CSD6 truth-facing grid. Each outer task had 168 candidate rows, giving 8064 candidate rows in total. This made it possible to compare

$$C_s(k, d) \qquad \text{with} \qquad R_s(k, d)$$

candidate by candidate.

The key finding was that low CV score did not always imply low truth RMSE. Some tasks had several candidates with nearly indistinguishable CV scores, while their truth RMSE values differed substantially. This supported the idea that exact CV-minimum selection may be too brittle, especially when the winning candidate lies on the edge of the grid or when many candidates are essentially tied in CV.

8.5 CSD9: offline robust replay rules

CSD9 replayed simple deterministic policies on the saved CSD8 surfaces. No models were refit. The baseline was exact CV minimum:

$$(\widehat{k}_s, \widehat{d}_s) = \operatorname*{arg\,min}_{(k,d)\in\mathcal{G}_{\text{full}}} C_s(k, d).$$

The most successful replay rule was the one-percent low-dimension rule. It first formed the near-tie set

$$T_s(0.01) = \{(k, d) \in \mathcal{G}_{\text{full}} : C_s(k, d) \le 1.01 \min_{(k',d')\in\mathcal{G}_{\text{full}}} C_s(k', d')\}.$$

If non-boundary candidates existed inside this set, boundary candidates were discarded. Among the remaining candidates, the rule preferred smaller d, then k closest to the middle of the support grid, then smaller CV score.

Table 3: CSD9 policy replay summary. The best replay was the one-percent low-dimension rule, but the gain over exact CV minimum was modest. A severe miss means selected truth ratio above 1.5.

Policy	Median truth ratio	Severe misses	Boundary rate	Median k, d
1% low-d	1.252	15	0.417	$(19.5, 6)$
CV minimum	1.271	16	0.521	$(19, 6)$
low-d penalty	1.284	17	0.271	$(19, 5.5)$
boundary penalty	1.284	17	0.292	$(20, 6)$
3% large-k	1.328	17	0.271	$(21.5, 6)$
3% low-d	1.336	18	0.271	$(19, 5)$
5% low-d	1.359	17	0.271	$(21, 5)$

CSD9 therefore gives a useful but deliberately cautious recommendation. The one-percent low-dimension rule is worth testing prospectively. It should not yet replace the current selector by default, because it was chosen after seeing the saved CSD8 surface and the paired median deltas were often exactly zero because many replay rules selected the same candidate as exact CV minimum.

9 Current interpretation

The CSD work clarifies the current taxonomy of dimension and support policies. `auto` is a global automatic dimension policy. `local.auto` is an anchor-specific automatic dimension policy. `full_kd` is a global full-grid numeric selector over (k, d). `sparse_kd` is a cheaper global numeric selector over a sparse subset of that grid.

The phrase "full grid" should therefore be read carefully. It means full in the candidate-grid sense:

$$\mathcal{G}_{\text{full}} = \{15, \ldots, 35\} \times \{1, \ldots, 8\}.$$

It does not mean full in the sense of allowing every anchor to choose its own (k_a, d_a). An anchor-specific coupled selector would be a different method:

$$a \mapsto (k_a, d_a).$$

That method is conceptually attractive, but it is not what CSD5–CSD9 tested.

10 Recommended next step

The next useful step is a prospective CSD10 validation run. It should compare at least three policies on the same outer tasks:

exact CV minimum over $\mathcal{G}_{\text{full}}$,

the 1% low-dimension near-tie replay rule made prospective,

a sparse-grid approximation.

The one-percent rule must be implemented inside the selector, not replayed after the fact. The report should preserve the candidate-level inner-CV surface and truth-facing outer scores so that the same diagnostic questions remain auditable.

A later, larger methodological direction is an anchor-specific coupled scale-dimension field:

$$a \mapsto (k_a, d_a).$$

That would be closer in spirit to `local.auto`, but it would require new regularization or stabilization machinery. It should not be described as the current `full_kd` method.

Artifact pointers

The main artifacts summarized here are:

- CSD plan: `~/current_projects/geosmooth/dev/methods/lps/specs/coupled_support_chart_dimension_selection_plan_2026-07-08.md`.
- CSD5 report: `~/current_projects/geosmooth/dev/methods/lps/reports/csd5_coupled_kd_evaluation_20260708/csd5_coupled_kd_evaluation_report.html`.
- CSD6 report: `~/current_projects/geosmooth/dev/methods/lps/reports/csd6_expanded_relative_regret_20260708/csd6_expanded_relative_regret_report.html`.
- CSD7 report: `~/current_projects/geosmooth/dev/methods/lps/reports/csd7_task_failure_diagnostics_20260708/csd7_task_failure_diagnostics_report.html`.
- CSD8 report: `~/current_projects/geosmooth/dev/methods/lps/reports/csd8_candidate_cv_surface_audit_20260708/csd8_candidate_cv_surface_audit_report.html`.
- CSD9 report: `~/current_projects/geosmooth/dev/methods/lps/reports/csd9_robust_cv_selection_policy_audit_20260708/csd9_robust_cv_selection_policy_audit_report.html`.